

Range-Generated Submodule Membership over Composite Moduli

Timestamped p -Adic Echelon Tables with Lean 4 Verification

ASTRA4OI Project

16 September 2026 Technical manuscript

Abstract

Given vectors $a_i \in (\mathbb{Z}/m\mathbb{Z})^d$, range-generated submodule membership asks whether a target belongs to $\langle a_1, \dots, a_r \rangle$. Over composite moduli, nonunit pivots have nontrivial annihilator relations, so a direct adaptation of single-pivot field elimination is incomplete. We present a timestamped p -adic echelon-table algorithm. Modulo p^k , each new vector starts at most k nonbranching power chains, and displaced residuals are processed by timestamp priority. Cyclic-quotient leading positions and digit-representation cardinalities yield an invariant valid for all suffixes simultaneously. For positive dimension, dominant coordinate-update counts are $O(nkd^2)$ for preprocessing and $O(d^2)$ per query; a known factorization gives $O(nKd^2)$ and $O(wd^2)$ via the Chinese remainder theorem. We provide general Lean 4 proofs, a declaration-by-declaration concordance, and independent program tests. Inversions, valuations, heap operations, and integer bit costs are charged separately. Factorization is not assumed free, and the formal execution relations are not claimed to be a source-refined verification of the executable implementation.

Keywords: modular linear algebra; range queries; finite chain rings; digit bases; formal verification

1 Introduction and scope

Over a field, invertible pivots support coordinate-by-coordinate elimination. In $\mathbb{Z}/m\mathbb{Z}$, nonzero elements need not be units. For instance, modulo 4, the multiple $(0, 2)$ of $(2, 1)$ cannot be recognized by a naive structure that retains only a first-coordinate pivot. Also, the full sequence $1, 2 \pmod{4}$ generates 1, whereas its last element does not. Retaining only the latest original vector therefore fails to support suffix filtering.

We study append-only membership queries. Recovering coefficients, editing interior elements, and deleting elements are outside the interface. The method stores k valuation slots per coordinate, using timestamps to represent all suffixes. The contribution is a complete insertion and query algorithm, a cyclic-extension and nonbranching-chain proof, and a research artifact linking paper statements, Lean declarations, and reproducible experiments.

Related work and positioning. A single modular membership problem reduces to integer-lattice membership. Kannan and Bachem gave polynomial algorithms for Smith and Hermite normal forms [2], providing foundational methods for the static problem. Bases with coefficients restricted to digits over finite rings are not new; for example, Kuijper and Pinto use p -linear combinations and p -bases for ring convolutional codes [3]. We specify the finite-vector, leading-slot notion used here and focus on timestamped maintenance of all suffixes, rather than claiming novelty for digit bases themselves. The formalization uses Lean 4 [1] and Mathlib [4]. We do not claim known optimality or infer comparative speedups over normal-form algorithms from uncontrolled timings.

2 Notation, interface, and cost model

Let $m = \prod_{s=1}^w p_s^{k_s} > 1$ with distinct primes p_s , exponents $k_s \geq 1$, and $K = \sum_s k_s$. The factorization is supplied. First fix $R = \mathbb{Z}/p^k\mathbb{Z}$, with $n, d, k \geq 1$. Coordinates are indexed by $0, \dots, d-1$ and original vectors by times $1, \dots, n$.

Definition 1 (Interval submodule). For $1 \leq l \leq r \leq n$, define

$$M_{l,r} = \text{span}_R\{a_i : l \leq i \leq r\}.$$

A membership query must return YES exactly when $x \in M_{l,r}$.

Lean: `Paper.intervalSpan; Paper.prefixSource_span.`

A *coordinate update* is a coordinate scaling during normalization or a multiply-subtract update during row reduction. It may contain a constant number of scalar modular operations. We count these separately from inversions, valuations, heap comparisons, and input scanning. Slots use constant-access arrays and the priority queue uses a heap with logarithmic operation cost. Exact constants below refer to coordinate-update allowances; full scalar and bit costs are separate.

Definition 2 (Leading position and normalized slot). For nonzero $b \in R^d$, let j be its first nonzero coordinate and $v = v_p(b_j) \in \{0, \dots, k-1\}$. Define $\text{lead}(b) = (j, v)$ with lexicographic order. A normalized row for slot (j, v) has $b_i = 0$ for $i < j$ and $b_j = p^v$.

Lean: `PadicEchelon.HasLead; PadicEchelon.PivotRow.`

If the canonical representative of the leading coordinate is $p^v u$, then u is a unit; multiplication by u^{-1} normalizes the row. The reduction coefficient b_j/p^v uses exact integer division of the canonical representative, never the inverse of the nonunit p^v .

3 Digit bases and cyclic quotients

Definition 3 (p -digit basis). A normalized row family $\mathcal{B}$ with distinct leading positions is a p -digit basis of H if

$$\{0, \dots, p-1\}^{\mathcal{B}} \longrightarrow H, \quad (c_b)_b \longmapsto \sum_{b \in \mathcal{B}} c_b b$$

is bijective. The submodule H need not be free over R .

Lean: `DigitCardinality.CardinalBasis; DigitCardinality.CardinalBasis.mem_iff_digitRep.`

Lemma 1 (Injective digit representations). A normalized row family with distinct leading positions has exactly $p^{|\mathcal{B}|}$ distinct digit combinations.

Proof. Subtract two different combinations and take the least leading position (j, v) with a nonzero coefficient. That coefficient has absolute value less than p and is not divisible by p . Its contribution in coordinate j has exact valuation v . Later slots in the same coordinate contribute valuation at least $v+1$, and rows led by later coordinates contribute zero there. Cancellation is impossible. $\square$

Lean: `DigitCardinality.encode_injective; DigitCardinality.card_range_encode.`

Lemma 2 (Complete digit-basis queries). For a digit basis of H , the leading position of any nonzero member is a stored slot. Reducing by normalized pivots, rejecting at a missing slot, and accepting at zero is correct and takes at most d row reductions.

Proof. The first nonzero term in a digit expansion determines its leading position, so a member cannot stop at a missing slot. Subtracting arbitrary ring multiples of basis rows preserves membership. Reversing a successful reduction sequence proves sufficiency of YES. Every step clears the entire current coordinate and preserves the zero prefix; hence at most d reductions occur. $\square$

Lean: BasisQuery.exists_correct_answer;BasisQuery.Derivation.correct.

Lemma 3 (Power levels of a cyclic quotient). *For $H \leq R^d$ and $a \in R^d$, there is $0 \leq h \leq k$ with $\text{ord}(a + H) = p^h$. Moreover, $p^e a + H \neq 0$ exactly when $e < h$. Enlarging H can only decrease h .*

Proof. The quotient R^d/H is annihilated by p^k , so the element order divides p^k and is p^h . The coset is annihilated by p^e exactly when $p^h \mid p^e$. A larger denominator submodule gives a quotient of the previous quotient; element orders can only become divisors. $\square$

Lean: CyclicQuotient.exists_quotient_order_exponent;CyclicQuotient.nsmul_pow_ne_zero_iff;CyclicQuotient.quotient_exponent_anti.

Lemma 4 (Missing-slot lead of a coset). *After reduction against a digit basis of H to the first missing slot, a nonzero residual's leading position depends only on its coset. Multiplication by a unit preserves it; multiplication by p and renewed reduction strictly increases it, unless the coset becomes zero.*

Proof. If two missing-slot residuals in one coset have different leads, their difference belongs to H but retains the earlier missing lead, contradicting Lemma 2. Unit scaling preserves both the lead and missing-slot property. Multiplication by p increases its valuation or clears that coordinate; further reductions can only advance through more coordinates. $\square$

Lean: BasisQuery.Basis.missing_coset_lead_unique;BasisQuery.Derivation.p_nsmul_final_strictly_later.

Lemma 5 (Digit basis of a cyclic extension). *Suppose $\text{ord}(a + H) = p^h$. Choose one normalized missing-slot representative $b_e \equiv u_e p^e a \pmod{H}$ for every surviving level $0 \leq e < h$, with units u_e . Together with a digit basis of H , these rows form a digit basis of $H + Ra$.*

Proof. By Lemma 4, the new leading positions are strictly increasing and absent from the old basis. With s old rows, the combined $s + h$ rows yield p^{s+h} distinct digit combinations. All lie in $H + Ra$, whose cardinality is $|H|p^h = p^{s+h}$, so the injection is surjective. In particular, distinct surviving levels cannot compete for one missing slot. $\square$

Lean: ArbitraryExtension.basisFromSettledLayers;CyclicExtension.no_same_slot_of_missing_layers.

4 Timestamped algorithm

Slot $B[j][v]$ stores a normalized row, timestamp t , and source power level e . These labels record the quotient identity

$$b \equiv u p^e a_t \pmod{H_{t+1}^{(r)}}, \quad H_{t+1}^{(r)} = \text{span}_R\{a_{t+1}, \dots, a_r\}, \quad u \in R^\times. \quad (1)$$

The source level e is not the slot valuation v . Subtracting a newer row must retain the older time and level.

Each replacement continues only one old residual and does not generate another p -multiple of the new pivot. Once initialized, power-chain tasks are only reduced or transferred. The nonbranching condition is essential for the cost bound.

Algorithm 1 Query suffix $[l, r]$ of the current prefix**Require:** Current-prefix table B , left endpoint l , target $x \in R^d$

```

1: for  $j = 0, \dots, d - 1$  do
2:   if  $x_j = 0$  then
3:     continue
4:   end if
5:    $v \leftarrow v_p(x_j); b \leftarrow B[j][v]$ 
6:   if  $b$  is absent or  $b.t < l$  then
7:     return NO
8:   end if
9:    $x \leftarrow x - (x_j/p^v)b$  ▷ Coordinates modulo  $p^k$ 
10: end for
11: return YES

```

Algorithm 2 Append vector a_r at time r

```

1:  $Q \leftarrow \{(p^e a_r, r, e, 0) : 0 \leq e < k, p^e a_r \neq 0\}$ 
2: while  $Q \neq \emptyset$  do
3:   Pop  $(x, t, e, s)$  of greatest time, breaking ties by least level
4:   for  $j = s, \dots, d - 1$  do
5:     if  $x_j = 0$  then
6:       continue
7:     end if
8:      $v \leftarrow v_p(x_j); b \leftarrow B[j][v]$ 
9:     if  $b$  is absent then
10:      Normalize  $x$  and store it with  $(t, e)$ ; break
11:     else if  $b.t > t$  then
12:        $x \leftarrow x - (x_j/p^v)b$ 
13:     else if  $b.t < t$  then
14:      Normalize  $x$  and replace the slot by  $(x, t, e)$ 
15:       $y \leftarrow b - (b_j/p^v)x$ 
16:      if  $y \neq 0$  then
17:        Enqueue  $(y, b.t, b.e, j + 1)$ 
18:      end if
19:      break
20:     else
21:      Impossible by Lemma 7
22:     end if
23:   end for
24: end while

```

5 Insertion invariants and completeness

Fix the prefix r being completed, and let p^{h_t} be the order of $a_t + H_{t+1}^{(r)}$. Call (t, e) *protected* if $e < h_t$. Across the active table and pending queue, maintain:

- (I1) stored slots are distinct, and normalized rows are ordered by leading position;
- (I2) active keys (t, e) are unique, and every active row satisfies (I1);
- (I3) every protected key is present in the table or queue.

Lemma 6 (Terminal extraction). *If a table satisfies these conditions with an empty queue, then for every l the rows of time at least l form a digit basis of $M_{l,r}$. Time t retains exactly levels $0, \dots, h_t - 1$.*

Proof. Induct on timestamps in descending order. Suppose newer rows form a digit basis of $H = H_{t+1}^{(r)}$. Time- t slots are absent from the newer table, and every $e < h_t$ occurs by (I3). An extra row with $e \geq h_t$ would belong to H by (I1) yet have a missing nonzero lead, contradicting Lemma 2. Protected keys are unique by (I2), so Lemma 5 gives the next extended digit basis. $\square$

Lean: TerminalExtraction.drained_table_correct.

Lemma 7 (Active invariants and collision exclusion). *Appending to a correct prefix table initializes (I1)–(I3), and each legal branch preserves them. A task popped at maximum timestamp cannot collide with a stored row of the same time and slot.*

Proof. Appending enlarges older H_{t+1} , so older protected-level sets can only shrink. All k new-time levels are initially queued, except zero rows, which cannot have protected keys. Thus (I3) holds initially. Normalization multiplies by a unit, and older rows only subtract newer rows, preserving quotient identities. Keys are moved, or dropped only with zero rows; the latter is impossible for a protected nonzero coset. Hence (I2)–(I3) persist, while replacement and empty-slot insertion preserve (I1).

Let the popped time be t . No newer task is pending, so all newer protected keys are already stored. Restricting the invariants to newer times permits Lemma 6, yielding a complete basis of H_{t+1} . If two time- t rows collide, their slot is missing from that newer basis. Their distinct keys imply distinct levels. A nonzero missing-slot residual cannot represent a vanished level, and distinct surviving levels have different leads by Lemma 5, a contradiction. This extracts the newer basis from local invariants; it does not assume that insertion has already completed successfully. $\square$

Lean: FastAppend.initial_protected;FastInvariant.scan_preservesKeysAndProtected;PrioritySafety.same_timestamp_collision_impossible.

Lemma 8 (Termination and nonbranching budgets). *An insertion always has a legal continuation and terminates. It visits at most kd columns, pops at most $k(d + 1)$ tasks, and has at most k pending tasks at any time.*

Proof. A nonzero row has a lead and admits unit normalization; Lemma 7 excludes the only collision error branch. Each initial task owns a column budget of d . Finishing removes a queue item; replacement adds only one successor starting at the next column and consumes at least one column. Queue length cannot increase, and total visits cannot exceed kd . The potential $|Q| + \sum_{q \in Q} (d - q.\text{start})$ decreases at every pop from an initial value at most $k(d + 1)$, proving termination and the pop bound. $\square$

Lean: Totality.appendResult_nonempty;WorklistComplexity.insertion_visits_le;WorklistComplexity.insertion_pops_le.

6 Main results and full cost accounting

Theorem 1 (Prime-power interval membership). *For every prime p , positive k, d , and input sequence, Algorithm 2, starting from the empty table, constructs at each right endpoint r a table valid for all intervals $[l, r]$. Algorithm 1 returns YES exactly when $x \in M_{l,r}$ and performs at most d row reductions. A table has at most dk rows. Processing n vectors uses at most $3nkd^2$ coordinate updates in the allowance for elimination and initial power-row preparation.*

Proof. The empty table satisfies the invariants. Lemmas 7 and 8 give legal terminating appends, and Lemma 6 restores all suffix bases. Prefix induction and Lemma 2 prove the answers. There are dk slots. At most two row passes occur per column visit, giving $2kd^2$ drain updates. One input-normalization pass and at most $k - 1$ passes multiplying by p use an additional allowance kd , for a total at most $3kd^2$ when $d \geq 1$. $\square$

Lean: Paper.theorem_1_prime_power.

Corollary 1 (Module cardinality). *If the filtered table has s rows, then $|M_{l,r}| = p^s$.*

Proof. Apply the digit-basis bijection and Lemma 1. $\square$

Lean: TimestampedBasis.CorrectTable.card_at;Paper.prefixSource_span.

Theorem 2 (CRT combination). *For a supplied factorization $m = \prod_{s=1}^w p_s^{k_s}$ into distinct primes, the full answer is the conjunction of component answers. Dominant preprocessing coordinate updates are $O(nkd^2)$, and each membership query uses $O(wd^2)$.*

Proof. Projecting a full linear expression proves necessity. Conversely, combine the component coefficients $\lambda_i^{(s)}$ by CRT separately for every original generator index i , obtaining one coefficient λ_i modulo m . Components need not first agree on integer coefficients. Summing their counters gives $3nKd^2$ and wd^2 respectively. $\square$

Lean: Paper.theorem_2_composite_interval;Paper.theorem_2_composite_preprocessing.

Proposition 1 (Implementation cost model). *Use array slots, a binary heap, and precomputed powers with binary-search divisibility tests for valuation. If scalar operations other than inversion are temporarily counted at unit cost and a unit inversion costs $I(p^k)$, then n appends and q queries cost*

$$O\left(nkd(d + \log(k + 1) + I(p^k)) + qd(d + \log(k + 1))\right). \quad (2)$$

Proof. Lemma 8 gives $O(kd)$ heap operations, each costing $O(\log(k + 1))$. Insertion has at most kd valuations and inversions; queries have at most d valuations and no inversions. Zero checks and coordinate searches cost at most $O(d)$ per pop and are absorbed. Precomputing powers takes $O(k)$, absorbed for positive n, d , as is the $O(n + q)$ cost of right-endpoint bucketing. $\square$

Lean verifies the event counts; array, heap, and scalar-subroutine cost contracts are part of the written analysis. Arbitrary-precision arithmetic requires separate bit costs for each operation, and factorization is additional. For $m = 1$, every valid query is YES but I/O and bookkeeping are not free. The program's `span_size` uses $O(dK)$ slot checks and exponentiation, not the membership-query bound.

Storage and historical intervals. The current table uses $O(Kd^2)$ coordinate cells. Each append creates at most $O(Kd)$ immutable row records. Saving dK references per prefix gives $O(nKd^2)$ total space and direct access to version r . This is a persistent-layout accounting argument, not allocator or byte-level memory verification.

7 Lean 4 concordance and trust boundary

Vectors are represented as functions from $\text{Fin } d$ to $\text{ZMod}(p^k)$; normalized leads, generated spans, and cardinality characterize digit bases. `Paper.prefixSource` labels the first r input vectors by $1, \dots, r$, and `Paper.prefixSource_span` equates timestamp filtering with the literal interval in Definition 1. Thus the machine statement of Theorem 1 is not merely about an assumed-correct abstract table.

The proof fields of `BuildResult` are established inductively, with both termination and query existence proved. `noncomputable` definitions and choice select mathematical evidence; they do not assert that compiled Lean definitions run with array/heap complexity. Increasing-level tie breaking is the reference program’s deterministic policy; the core proof only requires maximum timestamp selection and therefore covers that policy.

The table omits the shared namespace `CompositeModulusBasis`. Table 1 is generated from `paper/theorem-map.json`. Every listed declaration is compiled and its transitive axioms audited; the allowlist is `propext`, `Classical.choice`, and `Quot.sound`. Finite-model regression is separate and is not imported by the core entry point. Python source refinement, the general finite-ring reduction, and full bit complexity are outside the core machine theorems.

Table 1: Paper–Lean concordance.

Item	Claim	Lean declaration
Def. 1	Interval span and index bridge	<code>Paper.intervalSpan</code> <code>Paper.prefixSource_span</code> <code>Paper.compositePrefix_span</code>
Def. 2	Leading slots and normalized rows	<code>PadicEchelon.HasLead</code> <code>PadicEchelon.PivotRow</code>
Def. 3	Cardinality characterization of digit bases	<code>DigitCardinality.CardinalBasis</code> <code>DigitCardinality.CardinalBasis.mem_iff_digitRep</code>
Lem. 1	Injective digit encoding	<code>DigitCardinality.encode_injective</code> <code>DigitCardinality.card_range_encode</code>
Lem. 2	Query existence and both answer directions	<code>BasisQuery.exists_correct_answer</code> <code>BasisQuery.Derivation.correct</code>
Lem. 3	Quotient order and power levels	<code>CyclicQuotient.exists_quotient_order_exponent</code> <code>CyclicQuotient.nsmul_pow_ne_zero_iff</code> <code>CyclicQuotient.quotient_exponent_anti</code>
Lem. 4	Missing-slot leads and advancement under p	<code>BasisQuery.Basis.missing_coset_lead_unique</code> <code>BasisQuery.Derivation.p_nsmul_final_strictly_later</code>
Lem. 5	Cyclic extension and distinct layer slots	<code>ArbitraryExtension.basisFromSettledLayers</code> <code>CyclicExtension.no_same_slot_of_missing_layers</code>
Lem. 6	Extracting suffix digit bases	<code>TerminalExtraction.drained_table_correct</code>
Lem. 7	Protected keys and equal-time collision exclusion	<code>FastAppend.initial_protected</code> <code>FastInvariant.scan_preservesKeysAndProtected</code> <code>PrioritySafety.same_timestamp_collision_impossible</code>

Item	Claim	Lean declaration
Lem. 8	Totality, column budgets, and queue size	<code>Totality.appendResult_nonempty</code> <code>WorklistComplexity.insertion_visits_le</code> <code>WorklistComplexity.insertion_pops_le</code> <code>WorklistComplexity.insertion_queue_lengths_le</code>
Thm. 1	Prime-power interval theorem	<code>Paper.theorem_1_prime_power</code>
Cor. 1	Interval module cardinality	<code>TimestampedBasis.CorrectTable.card_at</code> <code>Paper.prefixSource_span</code>
Thm. 2	CRT interval queries and preprocessing	<code>Paper.theorem_2_composite_interval</code> <code>Paper.theorem_2_composite_preprocessing</code>
Prop. 1	Full implementation cost model (event counts only)	<code>VerifiedScan.Drain.initial_operational_bounds</code> <code>WorklistComplexity.insertion_heap_comparisons_le</code>

8 Experiments and reproducibility

The reference implementation uses only Python’s standard library. Independent answers come from three sources: coefficient enumeration for small moduli; triangularization and integer substitution in $\text{span}_{\mathbb{Z}}(a_l, \dots, a_r, mI_d)$ for large moduli; and gcd ideals under known invertible coordinate changes, plus one-dimensional gcd membership. The lattice oracle is first checked on 4,656 small targets and then on 9,484 additional large-modulus analytic targets.

Table 2: Program coverage. Membership assertions include checks across three APIs/stages.

Suite	Sequences	Intervals	Membership assertions
Small-modulus main tests	9,278	79,694	5,959,998
Additional large-modulus tests	78	9,332	240,420

Large tests use seed 20260915, with independently trial-verified primes up to 2, 147, 483, 647. They include 2^{64} , 3^{20} , $(10^9 + 7)^3$, and the 120-bit modulus $998244353^2 \cdot 1000000007^2$. Dimensions are 1, 2, 4, 8, 12 and lengths are 24 or 32. Data include zeros, duplicates, nonunits, coupled low-rank rows, and suffix rank drops. Of 80,140 large-modulus query cases, 46,386 are YES and 33,754 are NO; each is checked at append time, against historical versions, and through the offline API.

Performance samples use seed 120926 and $n = 4000$, $d = 16$, $q = 8000$, for modulus 72, $(10^9 + 7)^2$, and the above 120-bit CRT modulus. Each configuration independently spot-checks 83 answers. Timing covers solving only, excluding data generation and oracle checks. One run is not a statistical performance evaluation; we infer neither worst-case runtime, optimality, nor speedup over normal-form methods. Raw timings and counters are in `artifacts/benchmarks.txt`.

Artifact reproduction. The repository entry is <https://github.com/Zi-Gao/ASTRA4OI/tree/main/results/composite-modulus-basis>. Lean is pinned to 4.32.0, and Mathlib to v4.32.0 with locked dependency commits. From the result directory, `make test`, `make formal`, and `make paper` reproduce program tests, paper-declaration auditing, and bilingual PDFs. Logs are in `artifacts/`; setup and cache instructions are provided in the usage documentation.

9 Discussion and conclusion

The method organizes the closure requirements caused by nonunit pivots into finitely many levels of a cyclic quotient, using conservative timestamps to maintain every suffix simultaneously. Correctness rests on leading-position uniqueness, preservation of protected levels, and newer-time priority; complexity rests on initiating only k nonbranching chains per vector. Together these conditions establish a general result for all legal inputs, rather than an extrapolation from large-parameter tests.

Further questions include refined bit costs, factorization interfaces, coefficient recovery, updates or deletions, and controlled comparisons with other dynamic module algorithms. The additive-group reduction for general finite commutative rings is retained in supplementary documentation; its dimension expansion and formalization scope are separate from the main integer-modulus theorem.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [2] Ravindran Kannan and Achim Bachem. Polynomial algorithms for computing the Smith and Hermite normal forms of an integer matrix. *SIAM Journal on Computing*, 8(4):499–507, 1979.
- [3] Margreta Kuijper and Raquel Pinto. On minimality of convolutional ring encoders. <https://arxiv.org/abs/0801.3703>, 2009. arXiv:0801.3703, version 2.
- [4] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020.